Lenguaje de Programación Ostia - Documentación Oficial

1. Introducción y Filosofía

Ostia es un lenguaje de programación moderno, expresivo y diseñado íntegramente en español. Su sintaxis limpia y natural está orientada tanto a facilitar el aprendizaje de la programación como a permitir el desarrollo rápido y estructurado de aplicaciones.

---

2. Soporte Nativo de Caracteres (Ñ y Acentos)

Pensado desde su concepción para el idioma español, **Ostia soporta de forma nativa el uso de la letra 'ñ' (y 'Ñ') así como todas las vocales acentuadas** (`á`, `é`, `í`, `ó`, `ú`, `ü`, `Á`, `É`, `Í`, `Ó`, `Ú`) tanto en el código fuente (identificadores, nombres de variables, funciones) como en las cadenas de texto:

```
// Uso válido de acentos y la letra 'ñ' en nombres de variables y funciones

variable númeroDeCédula = 12345

variable añoActual = 2026

funcion verificarEspañol(texto mensaje) -> booleano {

    imprimirln("Procesando consulta en castellano...")

    retornar verdadero

}
```

---

3. Soporte en Visual Studio Code (Autocompletado e IntelliSense)

Ostia cuenta con soporte completo para editores de código modernos, permitiendo una experiencia similar a la de lenguajes como Python o JavaScript en **Visual Studio Code (VS Code)**.

Características Soportadas:

- **Resaltado de Sintaxis (Syntax Highlighting):** Reconocimiento automático de palabras clave (`variable`, `constante`, `funcion`, `si`, `sino`, `mientras`), tipos de datos y operadores mediante gramáticas de TextMate.
- **Autocompletado Inteligente (IntelliSense):** Sugerencias automáticas al escribir para:
    - Palabras reservadas y estructuras de control.
    - Funciones de la biblioteca estándar (`imprimir`, `leerNumero`, etc.).
    - Métodos y constantes del módulo matemático (`Matematica.PI`, `Matematica.raiz`, etc.).
- **Fragmentos de Código (*Snippets*):** Atajos para generar rápidamente bloques de código (por ejemplo, escribir `si` y pulsar Tab para autocompletar la estructura condicional completa).

¿Cómo instalar el soporte en VS Code?

1. Instala el generador oficial de extensiones de VS Code (`npm install -g yo generator-code`).
2. Crea una extensión de soporte de lenguaje local (`yo code` -> *New Language Support*).
3. Configura el archivo `package.json` para registrar los archivos `.ostia` junto con su gramática y archivo de autocompletado en TypeScript (`CompletionItemProvider`).

---

4. Sintaxis Básica y Variables

Declaración de Variables

Las variables se declaran utilizando la palabra reservada `variable`. Ostia cuenta con inferencia de tipos estática/dinámica según el contexto.

```
variable nombre = "Ana"

variable edad = 20

variable altura = 1.75

variable activo = verdadero
```

Constantes

Para valores que no deben cambiar durante la ejecución, se utiliza `constante`.

constante IVA = 0.21

---

5. Entrada y Salida de Datos

Ostia ofrece funciones integradas y directas para interactuar con la consola de forma intuitiva:

Salida por Consola

- `imprimir(mensaje)`: Muestra el texto o valor en pantalla sin salto de línea adicional.
- `imprimirln(mensaje)`: Muestra el texto y añade un salto de línea al final.

imprimirln("¡Hola, mundo!")

Entrada de Datos por el Usuario

- `leer(mensaje)` o `leerTexto(mensaje)`: Muestra un mensaje opcional y captura una cadena de texto introducida por el usuario.
- `leerNumero(mensaje)`: Captura una entrada numérica y la convierte automáticamente al tipo numérico adecuado (entero o decimal).

variable usuario = leerTexto("Introduce tu nombre: ")

variable númeroFavorito = leerNumero("Introduce tu número favorito: ")

imprimirln("Hola " + usuario + ", tu número al cuadrado es: " + (númeroFavorito * númeroFavorito))

---

6. Tipos de Datos Principales

- **`entero`**: Números sin decimales (ej. `42`, `-5`).
- **`decimal`**: Números con coma flotante (ej. `3.1416`, `-0.001`).
- **`texto`**: Cadenas de caracteres entre comillas dobles (ej. `"Ostia"`).
- **`booleano`**: Valores lógicos `verdadero` o `falso`.
- **`nulo`**: Ausencia de valor (`nulo`).

---

7. Estructuras de Control

Condicionales (`si`, `sino_si`, `sino`)

Permiten tomar decisiones basadas en condiciones booleanas.

```
variable calificación = leerNumero("Introduce tu nota: ")

si calificación >= 5 {

    imprimirln("¡Has aprobado!")

} sino_si calificación >= 4 {

    imprimirln("Estás en recuperación.")

} sino {

    imprimirln("Has suspendido. ¡Ánimo!")

}
```

Bucles e Iteraciones

Bucle `mientras` (While)

```
variable contador = 1

mientras contador <= 5 {

    imprimirln("Contador: " + contador)

    contador = contador + 1

}
```

Bucle `para` (For)

```
para variable i = 1 hasta 10 con_paso 1 {

    imprimirln("Iteración número: " + i)

}
```

---

8. Funciones y Modularidad

Las funciones se definen con la palabra clave `funcion`, especificando opcionalmente el tipo de retorno con `->`.

```
funcion calcularÁreaCírculo(decimal radio) -> decimal {

    retornar Matematica.PI * Matematica.potencia(radio, 2)

}

variable radioUsuario = leerNumero("Introduce el radio del círculo: ")

variable área = calcularÁreaCírculo(radioUsuario)

imprimirln("El área del círculo es: " + área)
```

---

9. Biblioteca Estándar y Módulo Matemático (`Matematica`)

El módulo `Matematica` incluye constantes universales y funciones avanzadas listas para usar.

Constantes Matemáticas

- **`Matematica.PI`** (o **`pi`**): 3.141592653589793
- **`Matematica.E`** (o **`e`**): 2.718281828459045
- **`Matematica.TAU`**: 6.283185307179586 (2π)
- **`Matematica.PHI`**: 1.618033988749895 (Proporción Áurea)

Funciones Matemáticas

- **Trigonometría**: `Matematica.seno(rad)`, `Matematica.coseno(rad)`, `Matematica.tangente(rad)`
- **Aritmética avanzada**:
  - `Matematica.raiz(x)`: Raíz cuadrada de un número.
  - `Matematica.potencia(base, exponente)`: Eleva la base al exponente indicado.
  - `Matematica.abs(x)`: Valor absoluto.
- **Redondeo y Utilidades**:

- `Matematica.redondear(x)`
- `Matematica.piso(x)`
- `Matematica.techo(x)`
- `Matematica.aleatorio()`: Genera un número decimal aleatorio entre 0 y 1.

---

11. Manejo de Excepciones y Errores

El lenguaje Ostia proporciona un mecanismo robusto y estructurado para la gestión y captura de errores en tiempo de ejecución mediante bloques de control y una jerarquía clara de excepciones.

Estructuras de Control de Errores

- **`intentar`**: Delimita el bloque de código que puede generar o provocar una excepción.
- **`capturar(Excepcion e)`**: Captura una excepción producida en el bloque `intentar` para gestionar el error adecuadamente.
- **`finalmente`**: Bloque opcional que se ejecuta siempre al terminar el bloque, sin importar si ocurrió un error o no.
- **`lanzar`** / **`lanza`**: Permite provocar manualmente una excepción explícita dentro del flujo del programa.

Jerarquía de Excepciones Predefinidas

- **`Excepcion`**: Clase base de la cual heredan todas las excepciones del sistema.
- **`ExcepcionAritmetica`**: Producida por operaciones matemáticas no válidas (por ejemplo, división por cero).
- **`ExcepcionNula`**: Ocurre al intentar acceder o invocar métodos en una referencia con valor `nulo`.
- **`ExcepcionIndice`**: Generada al intentar acceder a un índice fuera de los límites válidos de un arreglo o lista.
- **`ExcepcionFormatoNumero`**: Ocurre al intentar convertir una cadena no numérica a un tipo entero o decimal.
- **`ExcepcionEjecucion`**: Error genérico de tiempo de ejecución no clasificado en los tipos anteriores.

Métodos de los Objetos de Excepción

- **`obtenerMensaje()`**: Retorna el texto o descripción asociada al error capturado.
- **`obtenerTipo()`**: Retorna la clase o nombre del tipo de excepción lanzada.
- **`obtenerLinea()`**: Devuelve el número de línea exacto del código donde ocurrió el fallo.
- **`aCadena()`**: Genera una representación en texto completa del objeto de excepción.

Ejemplo de Uso

```
funcion dividir(entero a, entero b) -> entero {
    si b == 0 {
        lanza nueva ExcepcionAritmetica("No es posible dividir por cero")
    }
    retornar a / b
}

intentar {
    variable resultado = dividir(10, 0)
    imprimirln("Resultado: " + resultado)
} capturar(ExcepcionAritmetica error) {
    imprimirln("Error detectado: " + error.obtenerMensaje())
} finalmente {
    imprimirln("Operación finalizada.")
}
```

---

12. Ejemplo Completo: Adivina el Número

```
imprimirln("=== JUEGO: ADIVINA EL NÚMERO ===")

variable secreto = Matematica.redondear(Matematica.aleatorio() * 100) + 1

variable intento = 0

variable acierto = falso

mientras no acierto {

    intento = leerNumero("Adivina un número entre 1 y 100: ")



    si intento == secreto {

        imprimirln("¡Felicidades! Has acertado el número secreto.")

        acierto = verdadero
```

```
    } sino_si intento < secreto {
        imprimirln("El número secreto es mayor.")
    } sino {
        imprimirln("El número secreto es menor.")
    }
}
```